

Stream Ciphers, Pseudorandom Permutations, and Modes of Operation

CS 171: Cryptography

Sanjam Garg

September 17, 2026

Overview

- 1 Under the Hood
- 2 Stream Ciphers
- 3 Pseudorandom Permutations
- 4 Modes of Operation
- 5 Summary

Under the Hood

- So far: pseudorandom generators, pseudorandom functions, and CPA-secure encryption — treated as **abstract** primitives.
- Today: how are these primitives *actually* built and used in practice?
- We care about **concrete security**, and not just asymptotic security.
- **Efficiency**: keys of length n should give security against adversaries running in time $\approx 2^n$.

Caveat

These constructions are **heuristic**: their security is based on resistance to known attacks and extensive cryptanalysis, not on a reduction to a lower-level assumption.

Stream Ciphers

Idea

A **stream cipher** is a practical instantiation of a pseudorandom generator: from a short seed it produces a long pseudorandom stream of bits, generated *on demand*.

- Defined as a pair of (deterministic) algorithms (Init, GetBits).
- Encryption: XOR the message with the output stream (as in a stream-cipher / one-time-pad style scheme).

Stream Ciphers: Syntax

Definition 1 (Stream Cipher)

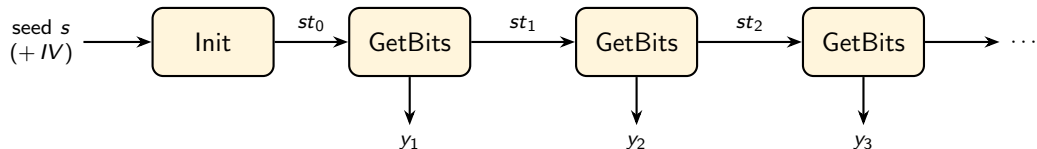
A stream cipher is a pair of deterministic algorithms (Init, GetBits):

- Init (initialization):
 - **Input:** a key (seed) s and an optional initialization vector IV .
 - **Output:** an initial state st_0 .
- GetBits (bit generation):
 - **Input:** the current state st .
 - **Output:** a next bit y and an updated state st' .

Generating a stream

Running GetBits repeatedly produces the desired number of output bits.

Stream Ciphers: Generating the Output



Usage

Use (Init, GetBits) to generate the desired number of output bits from the seed.

Stream Ciphers: Security

Without an IV

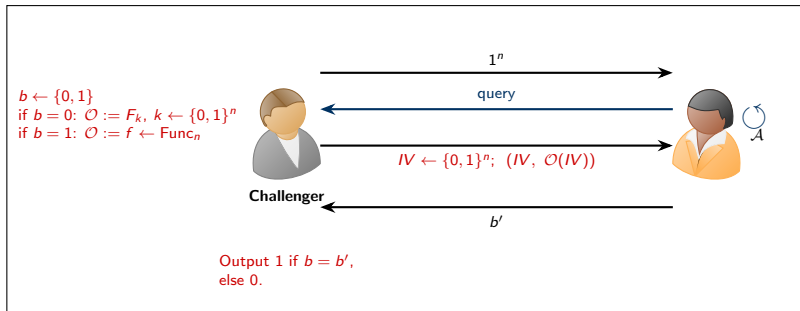
For a uniform key s , the output of GetBits should be a **pseudorandom** stream of bits. (This is exactly the PRG requirement.)

With an IV

For a uniform key s and uniform IV s (which are available to the attacker), the outputs of GetBits should be pseudorandom streams — one per IV . (This is a **weak PRF**: pseudorandom on uniformly random, attacker-visible inputs.)

wPRF _{$\mathcal{A}, F$} : Challenger vs. Adversary

wPRF _{$\mathcal{A}, F$} (n)



Weak PRF

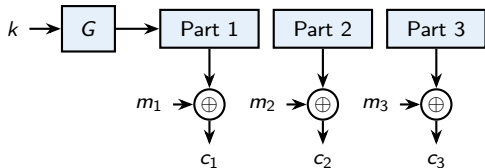
Same as PRF _{$\mathcal{A}, F$} (n), except that $\mathcal{A}$ does not choose the inputs: each query returns $(IV, \mathcal{O}(IV))$ for a fresh uniform IV .

F is a **weak PRF** if for all PPT $\mathcal{A}$: $\Pr[\text{wPRF}_{\mathcal{A}, F}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

Stream-Cipher Mode of Operation

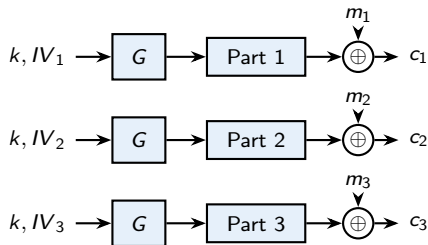
Synchronized mode

No IV; sender and receiver keep a shared, synchronized state.



Unsynchronized mode

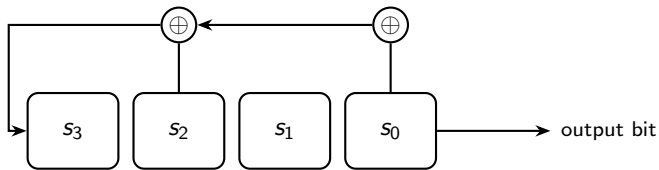
Each message uses a fresh IV; G is used as a **weak PRF** whose output is expanded.



- Communicate the IV alongside the ciphertext.

LFSRs (Linear Feedback Shift Registers)

- A degree- n LFSR has n registers.
- $s_{n-1} \cdots s_0$ are the contents of the registers.
- $c_{n-1} \cdots c_0$ are the **feedback coefficients**.



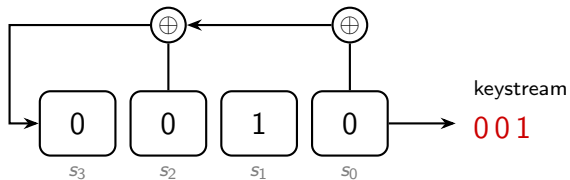
- Registers are updated in each clock cycle:

$$s'_{n-1} := \sum_j c_j s_j \bmod 2, \quad s'_i := s_{i+1} \text{ for } i \leq n-2.$$

LFSR: Example

A degree-4 LFSR with taps at positions 2 and 0: feedback $s'_3 = s_2 \oplus s_0$, output = s_0 .

$$s_2 \oplus s_0 = 1 \oplus 1 = 0$$



State trace (initial 0100)

state	output
0100	—
1010	0
0101	0
0010	1

- State = $s_3s_2s_1s_0$. Each clock cycle: emit s_0 , shift right, and set the new left bit to $s_2 \oplus s_0$.

Quest for a Good LFSR

- Intuitively: should cycle through all $2^n - 1$ non-zero states — otherwise the output bits start to repeat after a *short cycle*.
- It is known how to set the feedback coefficients to get such an LFSR (also called a **maximum-length** LFSR).
- A maximum-length LFSR has good statistical properties — but is **not** cryptographically secure.

Can you see an attack?

Attacks on LFSRs

- If the feedback coefficients are **fixed** (and known to the attacker), then the first n output bits fix the key entirely.
- If the feedback coefficients are **unknown** (and derived from the key), then the first $2n$ output bits fix the key and the coefficients.
 - Linear algebra is very powerful: solve linear equations over $\mathbb{F}_2$ for the unknowns.

Example: the LFSR from before ($n = 4$)

Keystream $y_0y_1 \cdots y_7 = 00101000$.

- **Known taps:** $y_0y_1y_2y_3 = s_0s_1s_2s_3$, so the key (initial state) is $s_3s_2s_1s_0 = 0100$.
- **Unknown taps:** each new bit is $y_{t+4} = c_3y_{t+3} \oplus c_2y_{t+2} \oplus c_1y_{t+1} \oplus c_0y_t$, so for $t = 0, 1, 2, 3$:

$$\begin{aligned} 1 &= c_2, & 0 &= c_3 \oplus c_1, \\ 0 &= c_2 \oplus c_0, & 0 &= c_1. \end{aligned}$$

Hence $c_3c_2c_1c_0 = 0101$: taps at 2 and 0.

Lesson

Linearity is **bad** for pseudorandomness.

Nonlinear FSR

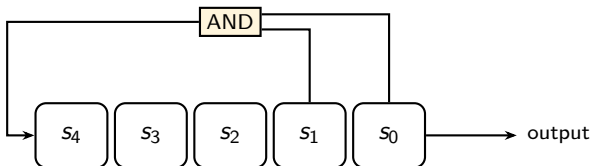
Adding non-linearity

- Make the **feedback** non-linear.
- Make the **output** non-linear.
- Use **multiple** LFSRs.
- **Mix** the above methods.

Goal

Allow for a long cycle *and* preserve the good statistical properties.

Nonlinear Feedback



- Non-linear feedback: $s'_4 = s_0 \wedge s_1$.
- **Is it secure?**
- Linear algebra is *not* useful any more!
- However, the AND **biases** the bits (its output is 1 only $\approx \frac{1}{4}$ of the time).
- How can we fix this?

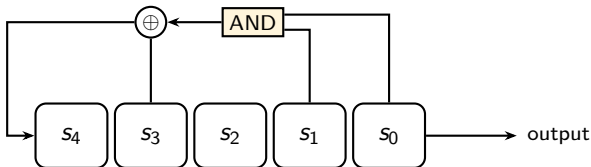
s_0	s_1	$s_0 \wedge s_1$
0	0	0
0	1	0
1	0	0
1	1	1

$$\Pr[s_0 \wedge s_1 = 1] = \frac{1}{4}$$

Nonlinear Feedback (Avoiding Bias)

Fix

Use of **XOR** helps remove bias!

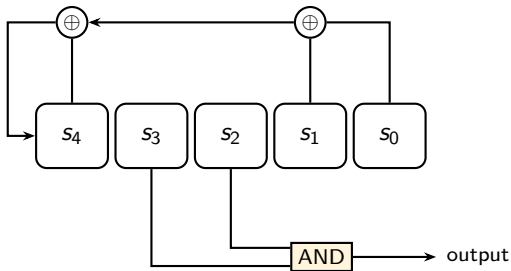


- Feedback $s'_4 = s_3 \oplus (s_0 \wedge s_1)$: XOR-ing the biased AND with a balanced state bit re-balances the feedback bit, while keeping the non-linearity.
- Treating s_3 as uniform and independent of s_0, s_1 :

$$\Pr[s'_4 = 1] = \underbrace{\Pr[s_3 = 1]}_{1/2} \cdot \underbrace{\Pr[s_0 \wedge s_1 = 0]}_{3/4} + \underbrace{\Pr[s_3 = 0]}_{1/2} \cdot \underbrace{\Pr[s_0 \wedge s_1 = 1]}_{1/4} = \frac{1}{2}.$$

Nonlinear Output

- Update of the LFSR state is **linear**, but the output is obtained as a **non-linear** function of the state.

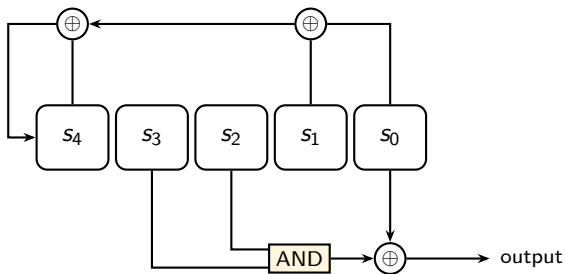


Again: we have a bias!

Nonlinear Output (Avoiding Bias)

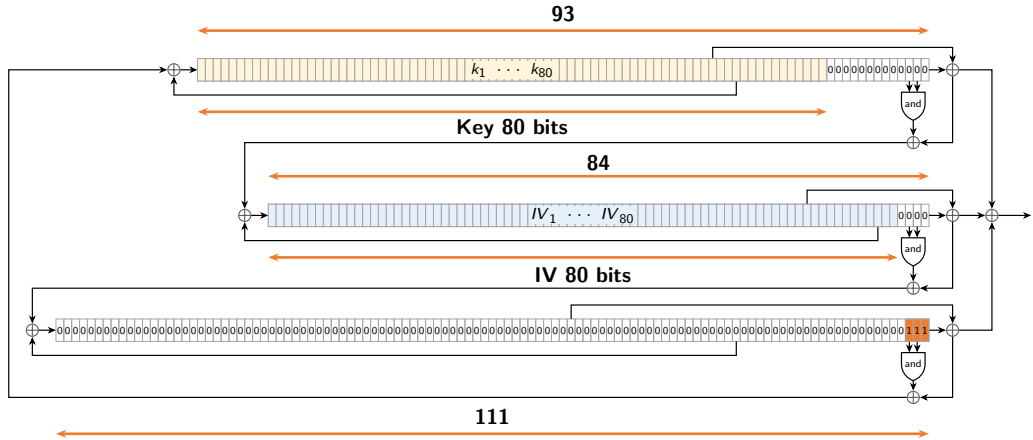
Fix

XOR the non-linear term with a state bit, exactly as before, to remove the bias: output $= (s_3 \wedge s_2) \oplus s_0$.



- Designed by De Cannière and Preneel in 2006 as part of the **eSTREAM** competition.
- Designed for **efficiency in hardware**.
- No attacks better than brute-force search are known!

Trivium: Structure



- Set everything else to 0, except the last three registers (of the last FSR), which are set to 1.
- Then, initialize by executing $4 \cdot 288$ times and discarding the output bits.

- Designed in 1987.
- Designed for efficiency in **software**, rather than hardware.
- No longer considered secure, but still interesting to study:
 - Simple description; **not** LFSR-based.
 - Still encountered in practice (WEP, 802.11).
 - Interesting attacks.

Set $S[i]$ to be the identity permutation of $\{0, \dots, 255\}$.

ALGORITHM 6.1

Init algorithm for RC4

Input: 16-byte key k

Output: Initial state (S, i, j)

(Note: All addition is done modulo 256)

for $i = 0$ to 255:

$S[i] := i$

$k[i] := k[i \bmod 16]$

$j := 0$

for $i = 0$ to 255:

$j := j + S[i] + k[i]$

 Swap $S[i]$ and $S[j]$

$i := 0, j := 0$

return (S, i, j)

Repeat the key to make it 256 bytes long.

One pseudorandom swap and obtain information for a pseudorandom location.

ALGORITHM 6.2

GetBits algorithm for RC4

Input: Current state (S, i, j)

Output: Output byte y ; updated state (S, i, j)
(Note: All addition is done modulo 256)

$i := i + 1$

$j := j + S[i]$

Swap $S[i]$ and $S[j]$

$t := S[i] + S[j]$

$y := S[t]$

return $(S, i, j), y$

Each entry of S is swapped with another pseudorandom entry of S .

RC4 Used with an Initialization Vector

- RC4 **was not designed** for that: there is no dedicated IV-handling step.
- Naive attempt: set the key to be $k = IV \parallel k'$ and run Init as usual.

Attack: Biased 2nd Output Byte

- Let S_t denote the state of the array S after t executions of GetBits; say S_0 is uniform, for simplicity.
- **Special case S:** $S_0[2] = 0$ and $S_0[1] = X \neq 2$.
- Then $\Pr[S] = \frac{1}{256} \cdot \left(1 - \frac{1}{255}\right) \approx \frac{1}{256}$.

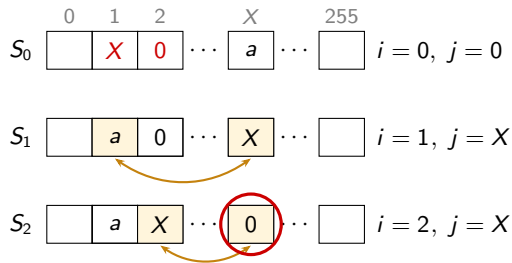
Attack: Biased 2nd Output Byte

In the special case S (with $a := S_0[X]$), run GetBits twice, starting from $i = j = 0$:

ALGORITHM 6.2

GetBits algorithm for RC4 (2nd call)

$i := i + 1$	$i = 2$
$j := j + S[i]$	$j = X + S_1[2] = X$
Swap $S[i]$ and $S[j]$	swap $S[2] \leftrightarrow S[X]$
$t := S[i] + S[j]$	$t = S_2[2] + S_2[X] = X$
$y := S[t]$	$y = S_2[X] = 0$
return $(S, i, j), y$	



Bias

Let y_2 be the 2nd output byte. Then

$\Pr[y_2 = 0] = \Pr[S] \cdot \Pr[y_2 = 0 \mid S] + \Pr[\neg S] \cdot \Pr[y_2 = 0 \mid \neg S] \approx \frac{1}{256} \cdot 1 + \left(1 - \frac{1}{256}\right) \cdot \frac{1}{256} \approx \frac{2}{256}$,
twice what a uniform stream would give.

RC4: More Attacks

- The second-byte bias is **already enough to break EAV-security** of Π :
 $c := m \oplus (\text{RC4 keystoream})$.

Adversary $\mathcal{A}$ in $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n)$

- Output m_0, m_1 whose 2nd bytes are 0x00 and 0xFF, respectively.
- Given c : output $b' := 0$ if the 2nd byte of c is 0x00, and $b' := 1$ otherwise.

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(n) = 1] = \frac{1}{2} \cdot \underbrace{\Pr[y_2 = 0x00]}_{\approx 2/256} + \frac{1}{2} \cdot \underbrace{\Pr[y_2 \neq 0xFF]}_{\approx 1 - 1/256} \approx \frac{1}{2} + \frac{1}{512}.$$

- **More serious attacks** arise when an IV is used (related keys $IV \parallel k$).
- Attacks can **recover keys in WEP**.

Takeaway

RC4 should not be used in new systems.

Pseudorandom Permutations / Block Ciphers

- What is a **permutation**?
 - A bijective function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$.
 - $\forall x \neq x': f(x) \neq f(x')$.
- Let Perm_n be the set of all permutations from n bits to n bits (compare: Func_n from the PRF definition).
- What is its size?
 - $|\text{Perm}_n| = (2^n)!$ (vs. $|\text{Func}_n| = 2^{n \cdot 2^n}$)

Pseudorandom Permutation (PRP): Definition

Game $\text{PRP}_{\mathcal{A},F}(n)$ (same as $\text{PRF}_{\mathcal{A},F}(n)$, with Perm_n in place of Func_n)

- ① $b \leftarrow \{0, 1\}$.
- ② If $b = 0$: $k \leftarrow \{0, 1\}^n$ and set $\mathcal{O} := F_k(\cdot)$; if $b = 1$: $f \leftarrow \text{Perm}_n$ and set $\mathcal{O} := f(\cdot)$.
- ③ $\mathcal{A}$ is given 1^n and **oracle access** to $\mathcal{O}(\cdot)$.
- ④ $\mathcal{A}$ outputs a guess b' .
- ⑤ Output 1 if $b = b'$, and 0 otherwise.

Definition 2 (Pseudorandom Permutation)

Let $F : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ be an **efficient, length-preserving, keyed permutation** (i.e. F_k is a bijection for every k). F is a **PRP** if for all PPT $\mathcal{A}$: $\Pr[\text{PRP}_{\mathcal{A},F}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

Every PRP is also a PRF! (A uniform permutation is hard to tell apart from a uniform function unless $\approx 2^{n/2}$ queries are made.)

Strong PRP

Game $\text{sPRP}_{\mathcal{A},F}(n)$

Same as $\text{PRP}_{\mathcal{A},F}(n)$, except that $\mathcal{A}$ is given oracle access to **both** $\mathcal{O}(\cdot)$ and $\mathcal{O}^{-1}(\cdot)$:

- if $b = 0$: $\mathcal{O} := F_k(\cdot)$ and $\mathcal{O}^{-1} := F_k^{-1}(\cdot)$; if $b = 1$: $\mathcal{O} := f(\cdot)$ and $\mathcal{O}^{-1} := f^{-1}(\cdot)$, for $f \leftarrow \text{Perm}_n$.

Definition 3 (Strong PRP)

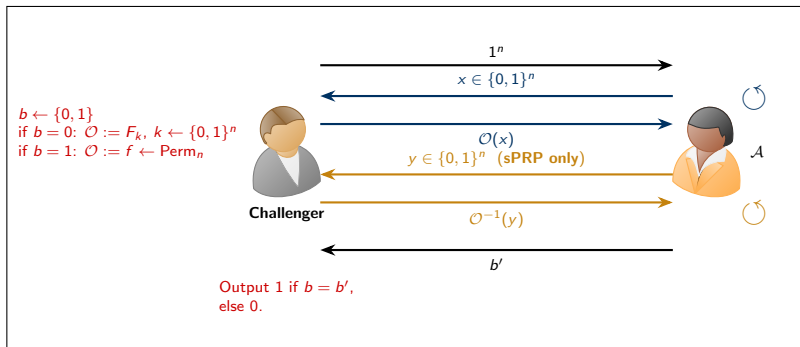
F is a **strong PRP** if for all PPT $\mathcal{A}$: $\Pr[\text{sPRP}_{\mathcal{A},F}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

Both computing **and** inverting!

- Block ciphers (e.g. AES) are designed to be strong PRPs. We use F_k to *encrypt* a block and F_k^{-1} to *decrypt* it.

PRP_{A,F} and sPRP_{A,F}: Challenger vs. Adversary

PRP_{A,F}(n) / sPRP_{A,F}(n)



Remark

In PRP_{A,F}, $\mathcal{A}$ may only evaluate $\mathcal{O}$. In sPRP_{A,F}, it may also invert it: F_k^{-1} in one world, f^{-1} in the other.

Electronic Code Book (ECB) Mode — Insecure

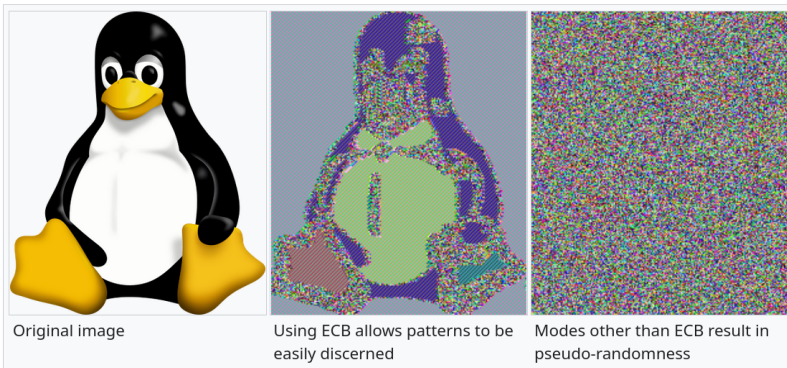


ECB

Each plaintext block is encrypted independently: $c_i := F_k(m_i)$. Decryption is done using F_k^{-1} .

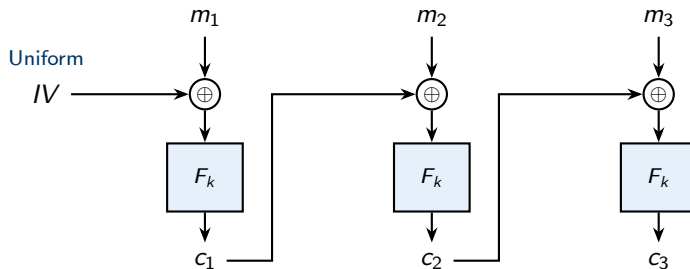
- **Not CPA-secure:** encryption is deterministic, so equal plaintext blocks give equal ciphertext blocks.

ECB is Visibly Insecure



- ECB leaks plaintext structure: identical blocks map to identical ciphertext blocks, so large-scale patterns remain visible.
- Source: https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation

Cipher Block Chaining (CBC) Mode

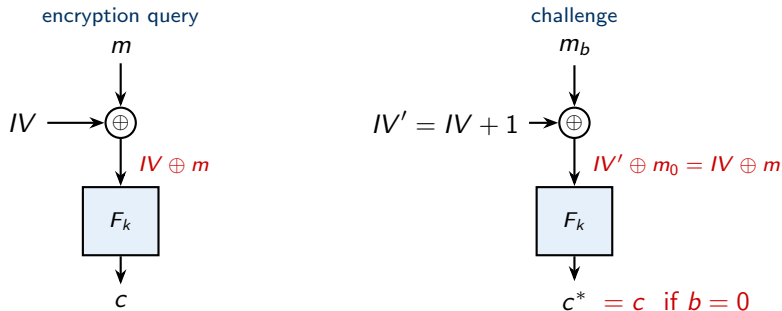


CBC

$c_0 := IV$ (uniform), and $c_i := F_k(m_i \oplus c_{i-1})$; decryption: $m_i := F_k^{-1}(c_i) \oplus c_{i-1}$. The IV is sent in the clear.

- CPA-secure when F_k is a PRP and the IV is freshly uniform per message.
- **Attack if the IV is not uniform** (but merely distinct across ciphertexts, e.g. a counter) — next slide.

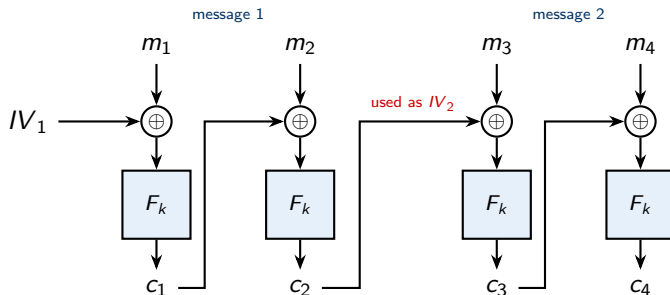
CBC with a Predictable IV Is Not CPA-Secure



- $\mathcal{A}$ asks for an encryption of m and gets (IV, c) , where $c = F_k(IV \oplus m)$.
- The IV is distinct but **predictable**: the challenge will use $IV' = IV + 1$.
- Challenge messages: $m_0 := IV' \oplus IV \oplus m$ and any $m_1 \neq m_0$.
- If $b = 0$, the input to F_k repeats, so $c^* = c$; if $b = 1$, it differs, so $c^* \neq c$ (F_k is a permutation).
- $\mathcal{A}$ outputs $b' := 0$ iff $c^* = c$, and wins with probability 1.

Is Chaining in CBC Mode Secure?

Chained IVs: reuse the last ciphertext block of one message as the IV of the next.

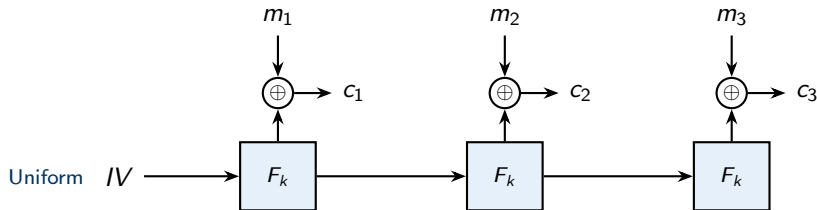


Not CPA-secure!

The adversary sees c_2 *before* choosing the next challenge. It can use the following challenge messages for m_3 : $IV_1 \oplus c_2 \oplus m_1$ and 0^ℓ .

- With the first choice, the input to F_k is $c_2 \oplus m_3 = IV_1 \oplus m_1$ — the same as for c_1 , so $c_3 = c_1$ and the adversary distinguishes.

Output Feedback (OFB) Mode



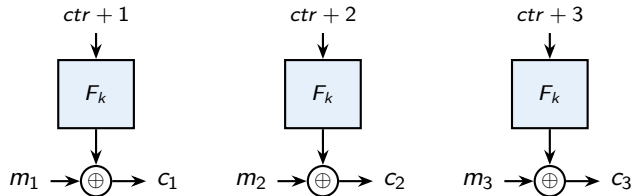
OFB

Pad stream: $r_0 := IV$ (uniform), $r_i := F_k(r_{i-1})$; then $c_i := m_i \oplus r_i$. The IV is sent in the clear.

- No need of F_k^{-1} — the pad is independent of the message, so a **PRF** suffices.
- **Positive:** all F_k evaluations can be made before the message is known. **Negative:** encryption and decryption are sequential.

Counter (CTR) Mode

Uniform ctr



- $c_i := m_i \oplus F_k(ctr + i)$, with addition mod 2^n ; the counter ctr is sent in the clear.
- Again no need of F_k^{-1} . **Positive:** easy to parallelize; possible to decrypt only the i -th block.

CPA-security (DIY)

Can be proved CPA-secure; the argument is similar to that for the PRF-based Π . We now need that the challenge values $ctr^* + 1, \dots, ctr^* + t^*$ are not used in any other adversarial query.

Summary

- Practical primitives are **heuristic**, judged by resistance to cryptanalysis; we care about **concrete** security (n -bit key $\Rightarrow$ time $\approx 2^n$).
- **Stream ciphers** (Init, GetBits) instantiate PRGs, or **weak PRFs** when used with an IV.
- **(Strong) PRPs**: indistinguishable from a uniform permutation (even given F_k^{-1}); every PRP is a PRF.
- **Modes of operation**: ECB is insecure; CBC needs a *uniform* IV (distinct or chained is not enough); OFB and CTR need only a PRF, and CTR parallelizes.